

Exercise 1 — Bank Customer Queue System (Queue / FIFO)

Customer class

- Fields: `customerId`, `name`, `serviceType` (e.g. "Deposit", "Withdrawal", "Loan"), `arrivalTime`.
- `arrivalTime` should be captured automatically (e.g. `LocalTime.now()`) at the moment the customer is enqueued, not typed in by the user — the customer's arrival is the moment they join the queue.

Storage

- `Queue<Customer>`, backed by `ArrayDeque<Customer>` — generally preferable to `LinkedList` for queue use in modern Java (better performance, no per-node object overhead), though `LinkedList` also implements `Queue` and would work.

Functions

- Add a new customer: `offer()`/`add()` onto the queue.
- Serve the front customer: `poll()` — check for `null` first (an empty queue) and print a "no customers waiting" message rather than crashing; `remove()` would throw instead of returning null, so `poll()` is the safer choice here.
- View the next customer without serving them: `peek()` — doesn't remove anything.
- Display the full waiting list: iterate the queue (its `iterator()` visits elements in the order they'd be served, front first).

Exercise 2 — Customer Support Ticket System (PriorityQueue)

SupportTicket class

- Fields: `ticketId`, `customerName`, `issueDescription`, `priority` (int, 1 = highest urgency, 5 = lowest), `status` (String, e.g. "OPEN" initially, updated to "RESOLVED" once processed).

Ordering

- Construct the `PriorityQueue<SupportTicket>` with a `Comparator` rather than making `SupportTicket` implement `Comparable` — that keeps the class itself free of one hard-coded "natural" ordering.
- `PriorityQueue` does **not** guarantee any particular order between elements that compare equal — two tickets with the same priority number have no promised relative order, unlike a stable sort. If tickets at the same priority should be handled in the order they were created, break ties on `ticketId` (or a separate insertion-order counter) as a secondary key in the comparator, e.g. `Comparator.comparingInt(SupportTicket::getPriority).thenComparingInt(SupportTicket::getTicketId)`.

Functions

- Create a new ticket: build a `SupportTicket` with status "OPEN" and add it to the queue.
- Process the ticket with the highest priority: `poll()` the queue (which returns the lowest priority number first, i.e. the most urgent), update its status, and display it.
- Display the waiting tickets in priority order: iterating a `PriorityQueue` directly does **not** guarantee priority order (only `poll()` does) — to list them in order without disturbing the queue, copy it into a new `PriorityQueue` (or a `List` you sort with the same comparator) and drain the copy.
- Show a count of tickets at each priority level: iterate the queue (order doesn't matter for counting) and tally by priority.

Exercise 3 — Reversing a String with a Stack

Requirements

- Push every character of the input string onto a stack, then pop them all off and append each to a result string — popping happens in the reverse order they were pushed, which is exactly what reverses the string.
- Use `Deque<Character>` (via `ArrayDeque`) as the stack rather than the legacy `java.util.Stack` class — `Stack` extends `Vector` and carries synchronization overhead that's rarely wanted; `ArrayDeque` is the modern recommended stack implementation, using `push()/pop()` the same way.

Example

- Input: `hello` → Output: `olleh`

Exercise 4 — Task Management Queue

Task class

- The original doesn't specify any fields at all for `Task` — a minimal reasonable shape is enough: `taskId` (or a name) and a `description`.

Functions

- Enqueue a new task (`offer()`).
- Dequeue and process the front task: `poll()` it, then display it ("processing" here just means handling and showing it — no further specific behavior is implied beyond that).
- View the task currently at the front, without removing it (`peek()`).

Exercise 5 — Parallel Task Processing Queue (Bonus)

Requirements

- Several worker threads pull requests from one shared queue and process them concurrently — that shared-queue-across-threads part is the key design constraint here.
- A plain `LinkedList` or `ArrayDeque` is **not** safe for concurrent access from multiple threads without external synchronization. Use `java.util.concurrent.BlockingQueue` (e.g. `LinkedBlockingQueue`) instead — it's built for exactly this producer/consumer pattern, with blocking `take()/put()` methods that handle the synchronization for you.
- A producer (the main thread, or a dedicated thread) enqueues requests; each of several worker threads loops calling `take()` and processing whatever it gets (e.g. `Thread.sleep()` to simulate work, then print which thread handled which request).
- Give the workers a way to stop: enqueue one sentinel "poison pill" value per worker once all real requests have been submitted; each worker exits its loop when it receives the sentinel instead of a real request.

Further extension: task priorities

- As a further extension on top of the above: swap the plain `BlockingQueue` for `java.util.concurrent.PriorityBlockingQueue` (the thread-safe counterpart to `PriorityQueue`) so higher-priority requests are handed to a free worker first.

Exercise 6 — Balanced Parentheses Checker

Algorithm

- Scan the expression left to right. On an opening bracket (({ [), push it. On a closing bracket, check the top of the stack: if the stack is empty, or the top isn't the matching opening bracket for this closer, the expression is unbalanced — stop and return `false`. Otherwise pop it and continue.
- Ignore every other character (letters, digits, operators) — only brackets affect the stack.
- After scanning the whole string, the result is `true` only if the stack ended up empty — a non-empty stack at the end means some opening bracket was never closed.

Examples

- `{[()()]}` → `true`
- `{[(())]}` → `false` (the closing `)` ends up matching against the `[` above it on the stack instead of the `(` that's actually waiting to be closed).
- `(( )` → `false` (an unclosed `(` remains on the stack at the end).

Exercise 7 — Infix to Postfix Conversion

Precedence

- `*` and `/` bind tighter than `+` and `-`; all four are left-to-right associative; parentheses override precedence as usual.

Algorithm (shunting-yard)

- Scan tokens left to right, using an operator stack and an output list:
- An operand (a variable or number) is appended straight to the output.
- `(` is pushed onto the stack.
- On `)`: pop operators off the stack into the output until a `(` is popped off — discard both parentheses, don't add either to the output.
- On an operator: while the stack's top is itself an operator with precedence $\geq$ the current one (and isn't a `(`), pop it into the output; then push the current operator.
- Once every token is scanned, pop any remaining operators off the stack into the output, in order.

Example

- Input: `A + B * (C - D)`
- Output: `A B C D - * +`

Exercise 8 — Implementing a Queue Using Two Stacks

Design

- Keep two stacks: `inStack` (for enqueue) and `outStack` (for dequeue).
- `enqueue(x)`: just push `x` onto `inStack`. Always $O(1)$.
- `dequeue()`: if `outStack` is empty, pour the entire contents of `inStack` into it (pop from `inStack`, push onto `outStack`, repeating until `inStack` is empty) — this pour reverses the order, which is exactly what turns `inStack`'s LIFO order back into correct FIFO order. Then pop from `outStack` and return that.

Why this is efficient overall

- A single `dequeue()` call can be $O(n)$ in the worst case, when it has to pour a full `inStack` into an empty `outStack`. But every individual element only ever gets poured from `inStack` to `outStack` once in its entire lifetime — so across any sequence of n enqueue/dequeue calls, the total pouring work is bounded by n , giving $O(1)$ amortized time per operation even though any single call's cost varies.

Exercise 9 — Palindrome Number Check Using a Queue

Requirements

- A negative number is never a palindrome — check the sign first and return `false` immediately if it's negative, before doing anything else.
- Split the (non-negative) number into its individual digits and load them into a `Deque<Integer>`, front to back in the order they appear. A plain `Queue` only exposes FIFO operations (removal from the front) — comparing the first and last digit against each other needs removal from *both* ends, which is exactly what the richer `Deque` interface adds on top of `Queue`.
- Repeatedly compare `pollFirst()` against `pollLast()`; if they ever differ, it's not a palindrome. Stop once the deque has 1 or 0 elements left — if every comparison matched, it is a palindrome.

Example

- Input: `12321` → Output: `true`
- Input: `12345` → Output: `false`

Exercise 10 — Warehouse Crate Stacking System (Capstone)

Rule 1 (a heavier crate can never sit on top of a lighter one) applies at *every point* during the program's run, not just to the final sorted result — that constraint shapes every algorithm below, the same way disk-size ordering shapes every move in the Tower of Hanoi puzzle this is modeled on.

Crate class

- `private String id;` (item code), `private int weight;`

Shelves

- Exactly 3 shelves, each a `Stack<Crate>` (or a `Deque<Crate>` used stack-style, via `ArrayDeque`, which is generally preferable to the legacy `java.util.Stack` class). Shelf 1 holds crates as they're initially entered; Shelf 2 is a working/temporary shelf used only during sorting and retrieval; Shelf 3 is where the fully sorted stack ends up.
- Also keep a separate registry, e.g. a `Map` from crate id to the index of the shelf it's currently sitting on — updated on every move — so "retrieve by code" doesn't have to blindly search all 3 shelves from scratch.

Adding crates

- Read a crate's id and weight from the keyboard and push it onto Shelf 1, in the order entered — no ordering is enforced at this stage; sorting happens as a separate step afterward.

Displaying shelf status

- For each shelf, print its crates from top to bottom — the first crate listed is the one that would be popped next; the last one listed is at the base of the stack.

Sorting (recursive, using all 3 shelves)

- Goal: move everything from Shelf 1 onto Shelf 3 so that Shelf 3 ends up with the heaviest crate at the bottom and the lightest on top, using Shelf 2 as a working area, and never violating Rule 1 at any intermediate step.
- This is a recursive "insertion sort via stacks" — similar in spirit to Hanoi's recursive peg-to-peg moves, though not the identical algorithm (Hanoi relocates an already-ordered stack; here the starting order is arbitrary and has to become sorted):
- Base case: if Shelf 1 is empty, there's nothing left to sort.
- Recursive case: pop the top crate off Shelf 1 and hold onto it (call it "current"). Recursively sort whatever remains on Shelf 1 first — after that call returns, Shelf 3 holds every other crate, already correctly sorted, and Shelf 2 is empty again. Now insert "current" into its correct spot: while Shelf 3 isn't empty and its top crate is lighter than current, pop that crate onto Shelf 2 to get it out of the way; once Shelf 3's top is heavier-or-equal (or Shelf 3 is empty), push current onto Shelf 3; then pour everything back from Shelf 2 onto Shelf 3 (pop, push, repeat) — the pour naturally restores the correct order, since it un-reverses the reversal from moving those crates onto Shelf 2 in the first place.

Retrieving a crate by code

- Look up which shelf the requested id is on via the registry. If it isn't currently on top of that shelf, pop off (and hold, on the spare shelf) every crate above it one at a time until it's exposed on top.
- Pop the target crate itself off — that's the one being retrieved and removed from the warehouse. Then pour the crates that were temporarily held back onto the same shelf, in their original relative order — since they were already validly ordered among themselves before being moved, and the shelf underneath was already validly sorted too, placing them back doesn't need any further re-sorting.

Saving and restoring shelf state

- Write the contents of all 3 shelves (which crates, on which shelf, top to bottom) to a .txt file, and provide a way to read that file back and rebuild the same shelf state — this needs to be a real, working feature of the program, not an afterthought.

Data structures

- At minimum: [Stack](#) (one per shelf) and [ArrayList](#) (a natural fit for the master crate registry, or for building the save-file's contents before writing them out).

Expected output

- Given crates P1 (5kg), P2 (3kg), P3 (2kg) starting on Shelf 1 with P1 on top:

```
1. Initial shelf state:
Shelf 1: [P1: 5kg, P2: 3kg, P3: 2kg]
Shelf 2: []
Shelf 3: []
```

```
2. After sorting:
Shelf 1: []
Shelf 2: []
Shelf 3: [P3: 2kg, P2: 3kg, P1: 5kg]
```

```
3. When retrieving crate P2:
Retrieve P2 from Shelf 3.
Shelf state:
Shelf 1: []
Shelf 2: []
Shelf 3: [P3: 2kg, P1: 5kg]
```

(Verified by hand: sorting [P1:5, P2:3, P3:2] with the algorithm above produces exactly this Shelf 3 result, and retrieving P2 — which sits underneath P3 — temporarily moves P3 aside and returns it directly on top of P1 afterward, matching the sample exactly.)

Exercise 11 — Basic LinkedList Operations

Setup

- An `LinkedList<Integer>` with several elements added to start.

Operations to implement

- Print all elements.
- Add an element at the beginning and at the end.
- Remove an element from the beginning and from the end.
- Insert an element at a given position.
- Search for an element.
- Calculate the sum of all elements.
- Reverse the list.
- Find the middle node — with `java.util.LinkedList`, the simplest approach is just `list.get(list.size() / 2)`, since `LinkedList` already implements the `List` interface; if you want to practice the classic slow/fast-pointer technique instead, you can walk it manually with two `ListIterators`, one advancing twice as often as the other.
- Merge two unsorted linked lists into one sorted linked list: since the inputs are explicitly unsorted (not the classic "merge two already-sorted lists" problem, which uses a different linear-merge technique that depends on both inputs already being in order), the straightforward approach is enough here — combine all elements from both lists into one list, then sort that combined list, e.g. with `Collections.sort()`.
- Remove duplicate elements from an already-sorted list: because it's sorted, every duplicate is guaranteed to sit immediately next to the element it duplicates, so a single pass comparing each element only to the one right before it is enough — no need for a `HashSet`-based approach, which would be doing more work than the sorted input actually requires.

Checking for a cycle

- `java.util.LinkedList` can't be made to contain a cycle through its public API at all — there's no way to point one node's internal `next` reference at an earlier node using `add()`/`remove()` and friends. Classic cycle-detection exercises assume a hand-built linked structure where that's possible.
- For this one operation specifically, build a minimal custom node class alongside your `LinkedList` work — e.g. `class Node { int val; Node next; }` — link a chain of them manually, and deliberately set the last node's `next` back to an earlier node to construct a test case with a real cycle.
- Implement Floyd's cycle-detection algorithm ("tortoise and hare") against that custom structure: two pointers start at the head, one advances one node per step and the other two nodes per step; if the fast pointer ever reaches the end (null), there's no cycle; if the two pointers ever point at the same node, there is one.